

Midterm Practice

EGCI341

Midterm

- Date: Th. 4-Jun-2026, 12:00-13:50
- Room: 2303
- 1-page, A4, single side cheat sheet (Write student ID on cheat sheet)
- 30% marks
- Do not forget student id card and student uniform

**** Any kinds of cheating will be reported to OAA and follows the MUIC's policy**

Code: EGCI341 section : 1
Course: Software Engineering
Credit: 4(4-0-8)

**Midterm
For Term
: 3/2025**

DISABLE EXAM

Date Time 04/06/2026 Time : 12:00 - 13:50

Status **Approved**

Exam Room Lecture Room 2303

Proctor # 2

Requests ☒ Lecturer Present

Student Remarks - 1-page A4 single side cheat sheet is allowed. - Write student id on your cheat sheet.

Topics

- CMMI and Agile Software Process Models (3 questions – short answers)
- Design Model and Functional Dependencies (4 main questions – short answers and draw diagrams)
- Architectural Design and C4 Model (2 questions – 1 short answer and diagrams)

Practice: Short Answers

- ~~■ What will be the obstacles when your software team needs to integrate the new software with legacy system.~~
- What is the duties of “Scrum Master”? And What are the questions that Scrum master asks during the daily meeting.
- Explain how to get product backlog in Scrum.
- ~~■ What kind of project that suitable for Kanban~~

Practice: Short Answers

```
1    public void main() {  
2        Person bob = new Person();  
3        String phone = bob.getPhone();  
4    }  
5    public class Person{  
6        public String getPhone(){  
7            return "0123456";  
8        }  
9    }  
10 }
```

- Where is the encapsulation in main()? (Which line no.)
- Where is the information hiding in main()? (Which line no.)

Practice: Short Answers

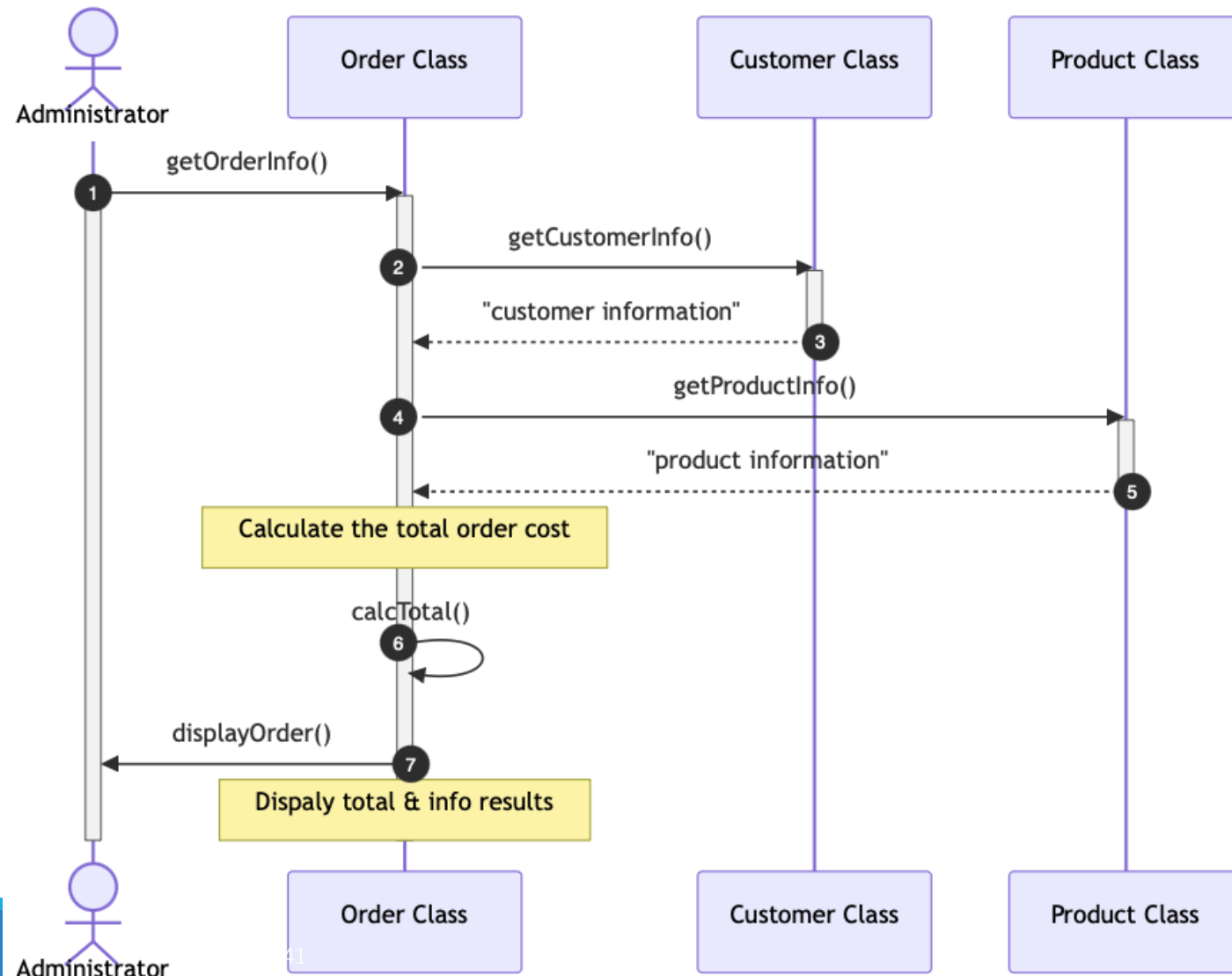
- What is the difference between aggregation relationship and composition relationship? Show the example.

Draw a sequence diagram

- Administrator want to retrieve order detail from one customer.
 - Administrator request “getOrderInfo()” from Order class.
 - Then “getOrderInfo()” will trigger “getCustomerInfo()” from Customer class and “getProductInfo()” from Product class to query the information
 - The responses from getCustomerInfo() is “customer information” and the response from getProductInfo() is “product information”.
 - The next step is to calculate the total by calling calcTotal() from Order class then return result to administrator by using displayOrder()

Draw a sequence diagram

Draw a sequence diagram



sequenceDiagram
autonumber
actor Admin as Administrator
participant O as Order Class
participant C as Customer Class
participant P as Product Class

Admin->>O: getOrderInfo()
activate Admin
activate O
%% Parallel/Sequential queries triggered by getOrderInfo()

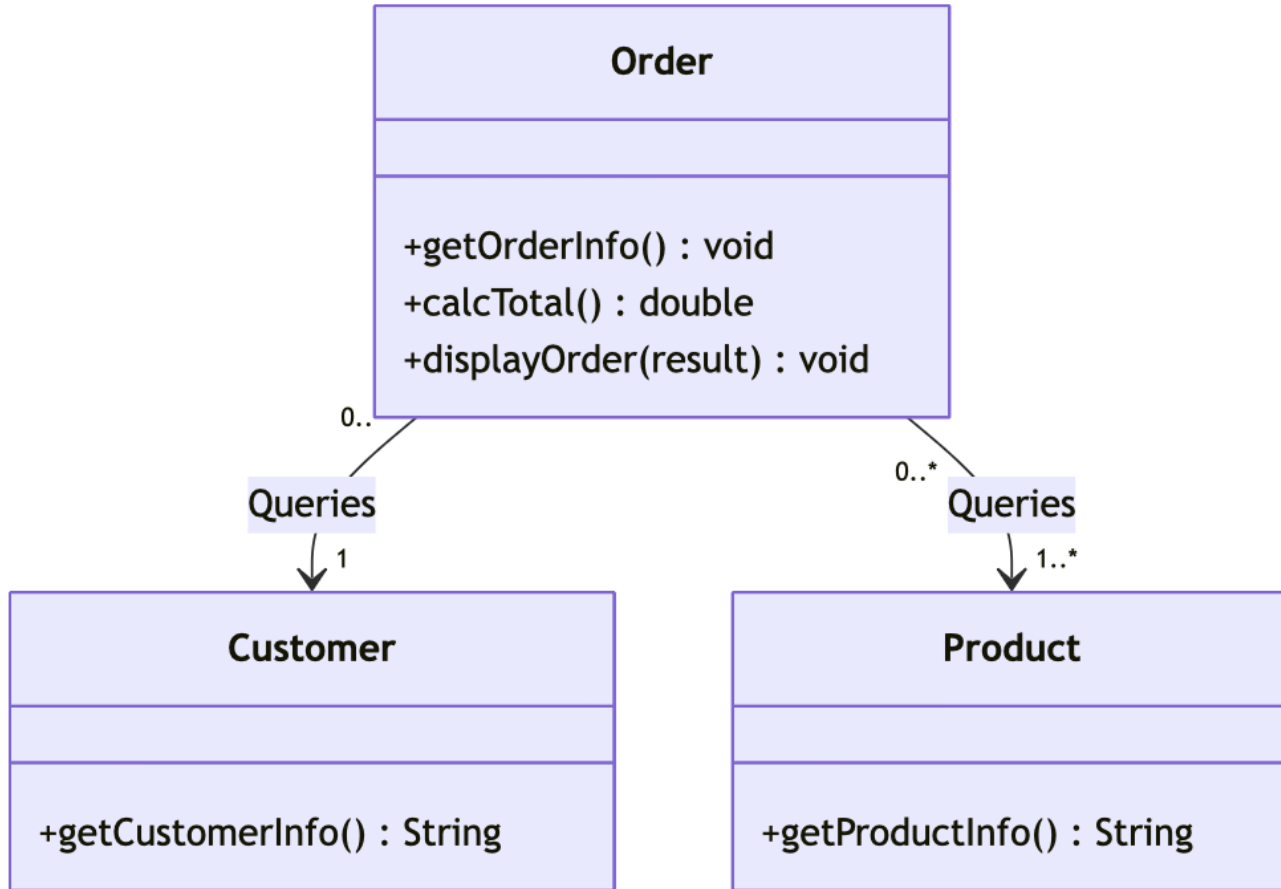
O->>C: getCustomerInfo()
activate C
C-->>O: "customer information"
deactivate C

O->>P: getProductInfo()
activate P
P-->>O: "product information"
deactivate P

%% Internal execution step
Note over O: Calculate the total order cost
O->>O: calcTotal()
%% Finalstep
O->>Admin: displayOrder()
Note over O: Display total & info results
deactivate O
deactivate Admin

According to previous sequence diagram, Draw class diagram

According to previous sequence diagram, Draw class diagram



```
classDiagram
direction TR
Order "0..*" --> "1" Customer : Queries
Order "0..*" --> "1..*" Product : Queries
```

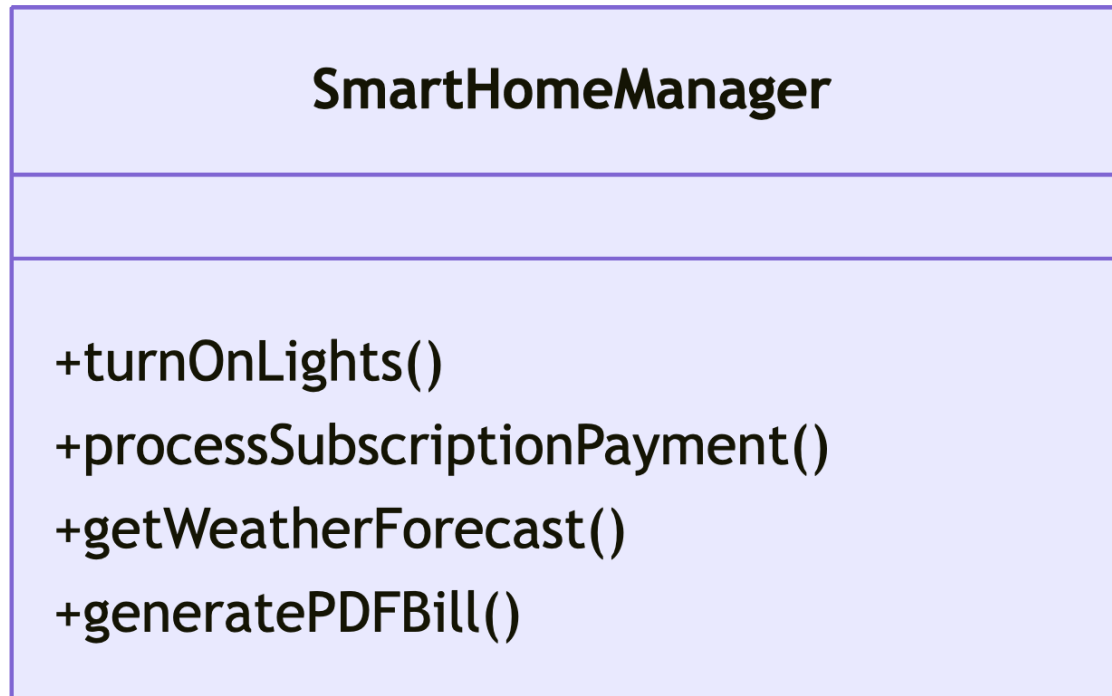
```
class Order {
+getOrderInfo() void
+calcTotal() double
+displayOrder(result) void
}
```

```
class Customer {
+getCustomerInfo() String
}
```

```
class Product {
+getProductInfo() String
}
```

Functional Dependencies

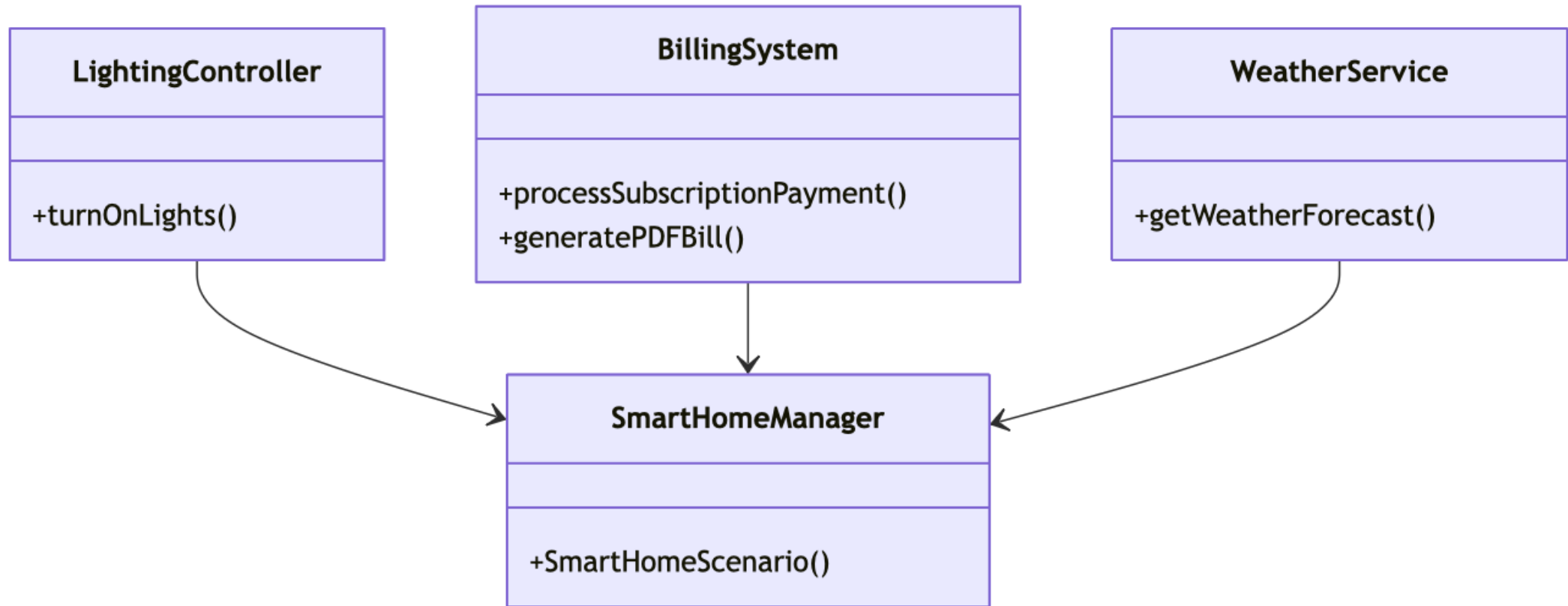
Low cohesion: Functions are unrelated



Functional Dependencies

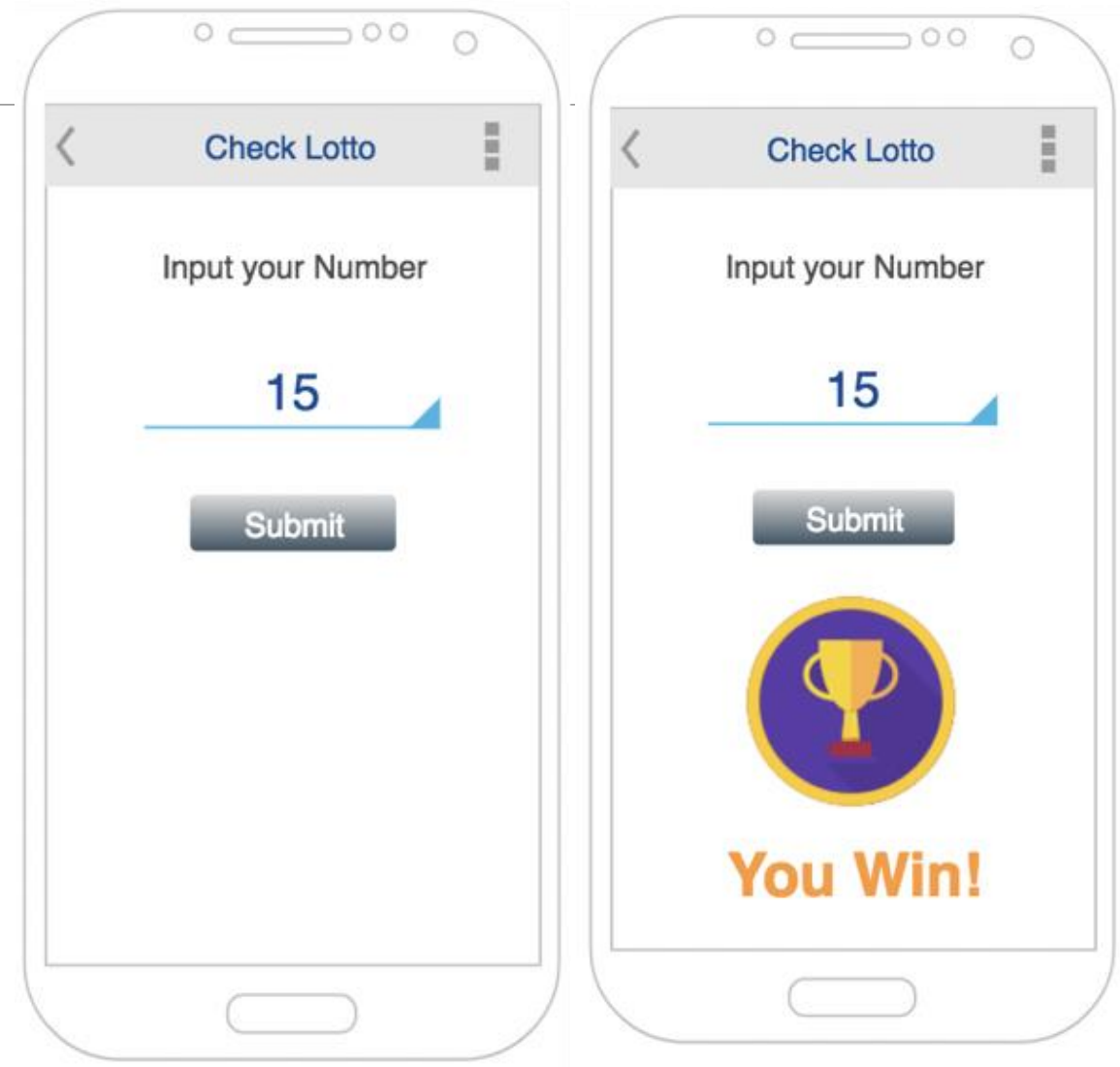
Functional Dependencies

High cohesion



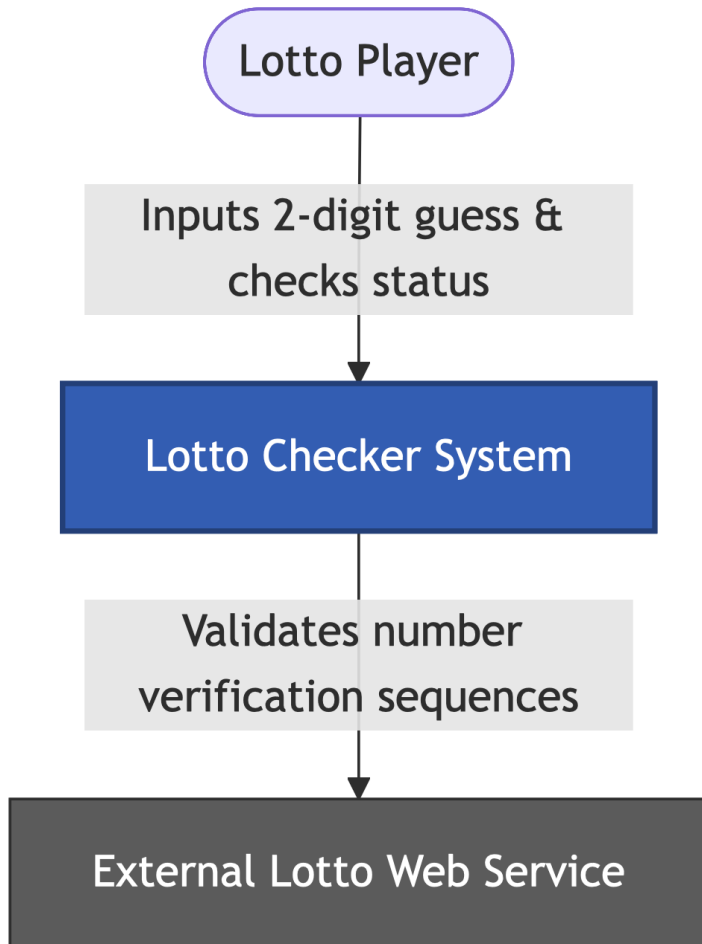
Software Architecture

- Design the mobile application architecture to check whether the input no. matches with one of the winning nos.
- To check a winning number, there is a web service that can receive the input as 2-digit number in form of JSON format such as {number: 15}
- The result returning from web service is also JSON format as {result: "win"} or {result: "lose"}.



Lotto Architectural Design : Context diagram

Lotto Architectural Design : Context diagram



flowchart TB

Player([Lotto Player]) --> |Inputs 2-digit guess & checks status| LCS[Lotto Checker System]

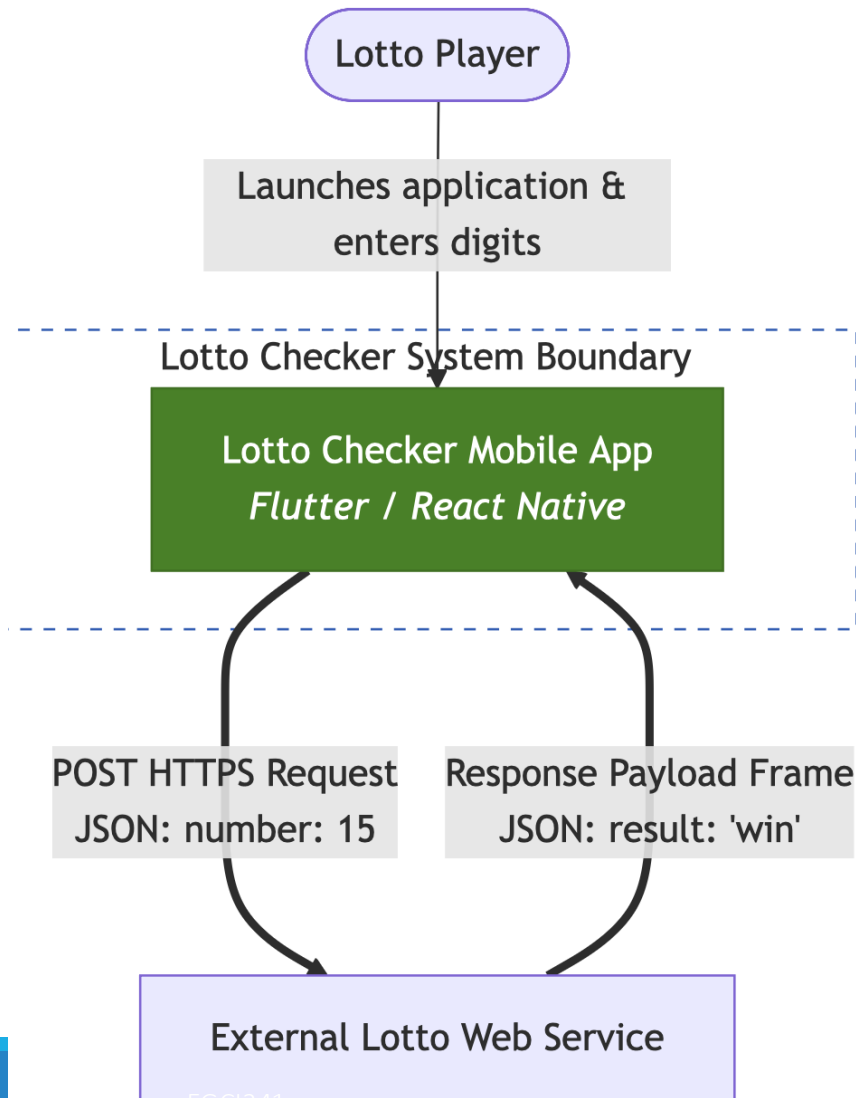
LCS --> |Validates number verification sequences| EWS[External Lotto Web Service]

style LCS fill:#1168bd,stroke:#0b4884,color:#fff,stroke-width:2px

style EWS fill:#666,stroke:#333,color:#fff

Lotto Architectural Design : Container diagram

Lotto Architectural Design : Container diagram



flowchart TB

subgraph LCS [Lotto Checker System Boundary]

App[Lotto Checker Mobile App
<i>Flutter / React Native</i>]

end

Player([Lotto Player]) --> |Launches application & enters digits| App

%% Outbound interface calls

App ==> |POST HTTPS Request
JSON: number: 15| EWS[External Lotto Web Service]

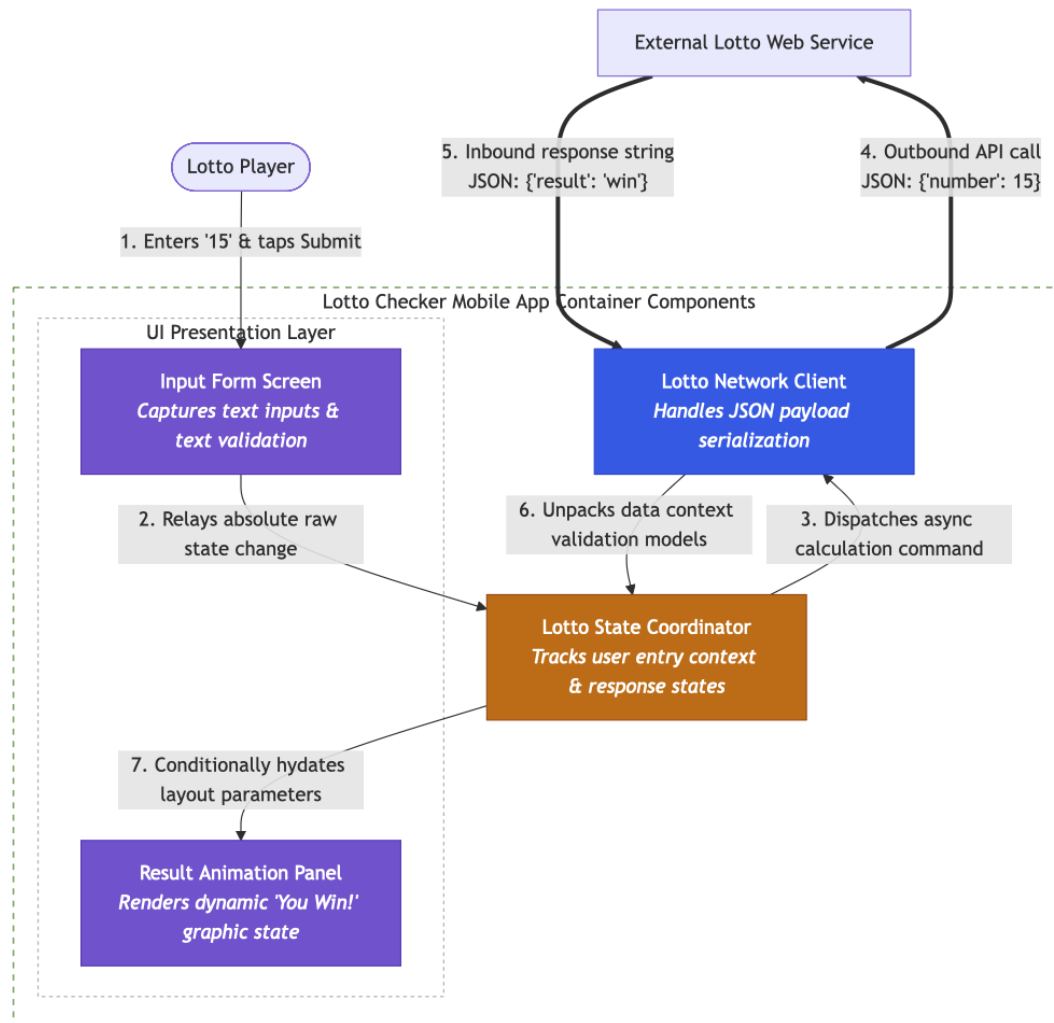
EWS ==> |Response Payload Frame
JSON: result: 'win'| App

style LCS fill:none,stroke:#1168bd,stroke-dasharray: 5 5

style App fill:#228b22,stroke:#1e7b1e,color:#fff

Lotto Architectural Design : Component diagram

Lotto Architectural Design : Component diagram



flowchart TB

ExtService[External Lotto Web Service]

subgraph AppContainer [Lotto Checker Mobile App Container Components]

subgraph UI [UI Presentation Layer]

InputForm[Input Form Screen
Captures text inputs & text validation</i>]

ResultPanel[Result Animation Panel
Renders dynamic 'You Win!' graphic state</i>]

end

StateMgr[Lotto State Coordinator
Tracks user entry context & response states</i>]

HttpService[Lotto Network Client
Handles JSON payload serialization</i>]

end

%% User Interaction Flow

Player([Lotto Player]) -->|1. Enters '15' & taps Submit| InputForm

InputForm -->|2. Relays absolute raw state change| StateMgr

%% Execution Controller Pipeline Flow

StateMgr -->|3. Dispatches async calculation command| HttpService

HttpService ==>|"4. Outbound API call
JSON: {'number': 15}"| ExtService

%% Return Trip Processing Flow

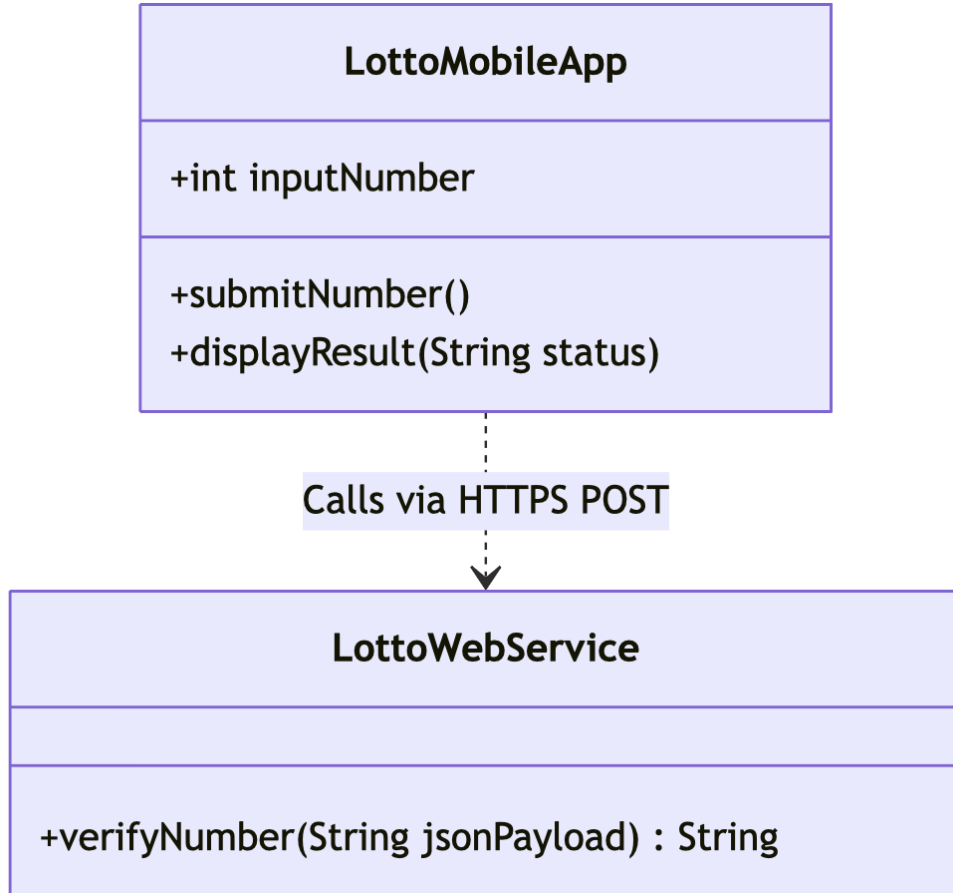
ExtService ==>|"5. Inbound response string
JSON: {'result': 'win'}"| HttpService

HttpService -->|6. Unpacks data context validation models| StateMgr

StateMgr -->|7. Conditionally hydrates layout parameters| ResultPanel

Draw Class Diagram

Draw Class Diagram



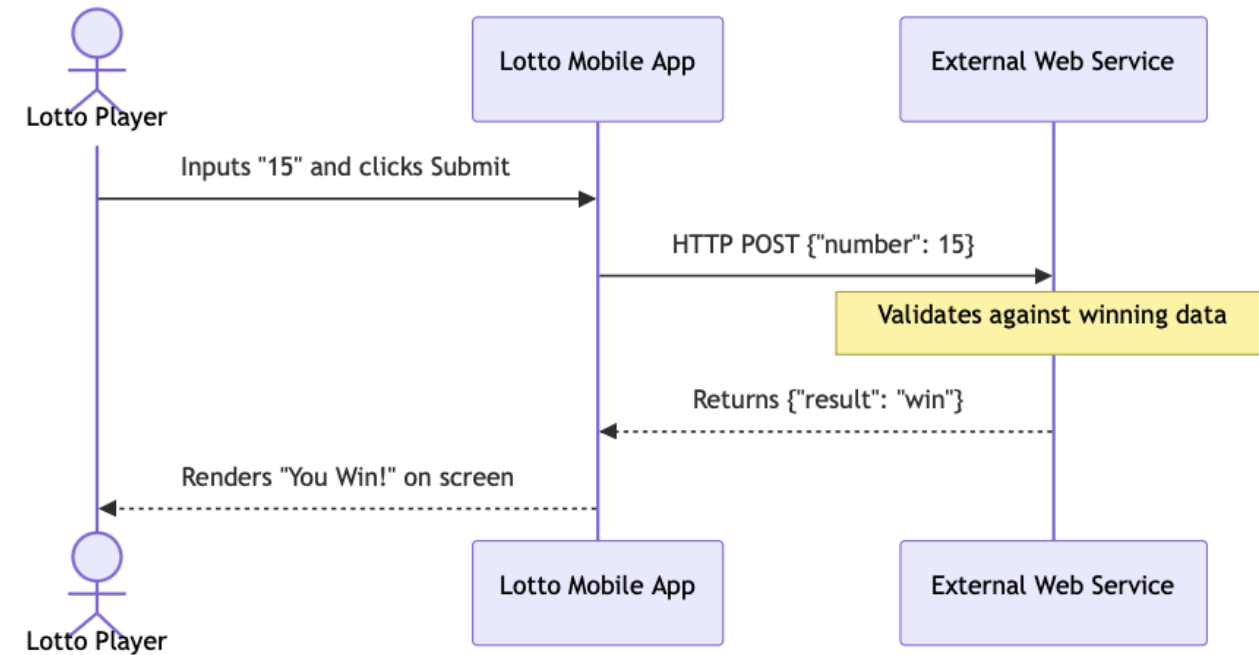
```
classDiagram
class LottoMobileApp {
+int inputNumber
+submitNumber()
+displayResult(String status)
}
```

```
class LottoWebService {
+verifyNumber(String jsonPayload) String
}
```

LottoMobileApp ..> **LottoWebService** : Calls via HTTPS POST

Draw Sequence Diagram

Draw Sequence Diagram



sequenceDiagram

actor Player as Lotto Player

participant App as Lotto Mobile App

participant API as External Web Service

Player->>App: Inputs "15" and clicks Submit

App->>API: HTTP POST {"number": 15}

Note over API: Validates against winning data

API-->>App: Returns {"result": "win"}

App-->>Player: Renders "You Win!" on screen